M2

Have you studied NumPy or Pandas?

A. Yes

B. No

What is the number of dimensions of the array arr?

Input your answer

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

- Number of dimensions of the NumPy array: ndim
- Shape of the NumPy array: shape
- Size of the NumPy array: size

What is the output?

Input your answer

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

print(arr[1, 1, 2])
```

What is the output?

Input your answer

```
a = np.array([1, 2, 3, 4])
print(a[a>2])
```

What is the output?

Input your answer

```
a = np.array([1,2,3])
b = a
a += 1
print(b)
```

What is the output?

Input your answer

```
a = np.array([1,2,3,4])
Slice = a[:2]
Slice[0] = 10
print(a)
```

What is the output?

Input your answer

```
import numpy as np
a=np.array([1, 4, 9, 16, 25, 36, 49, 64, 81])
b=np.array([[2, -2],[0, -1]])
print(a[b])
```

What is the output?

Input your answer

```
import numpy as np
a=np.array([1, 2, 3, 4, 5, 6])
print(a.reshape(3,2))
```

Series Labels

- If you specified labels for a Pandas Series, Use **Series**.loc attribute to select some values from the given Series object based on the labels.
- Pandas **Series.iloc** attribute enables purely integer-location based indexing for selection by position over the given Series object.
- Use **Series**[] to access elements by **labels (either index or keys)**

```
import pandas as pd
sr = pd.Series(['New York', 'Chicago', 'Toronto', 'Lisbon'])
sr.index = ['City 1', 'City 2', 'City 3', 'City 4’]

print('''print: sr.loc[['City 1', 'City 2']]''')
print(sr.loc[['City 1', 'City 2’]])

print('''print: sr.iloc[0:2]''')
print(sr.iloc[0:2])

print('''print: sr[0:2]''')
print(sr[0:2])

print('''print: sr[['City 1', 'City 2']]''')
print(sr[['City 1', 'City 2']])
```

```
print: sr.loc[['City 1', 'City 2']]
City 1    New York
City 2     Chicago
dtype: object
print: sr.iloc[0:2]
City 1    New York
City 2     Chicago
dtype: object
print: sr[0:2]
City 1    New York
City 2     Chicago
dtype: object
print: sr[['City 1', 'City 2']]
City 1    New York
City 2     Chicago
dtype: object
```

Access Elements of DataFrame

- Access a ***single row***
  - df.loc[] to access rows by *rowName*
  - df.iloc[] to access rows by row*Index*
- Access a ***single element***.
  - df.loc[rowName, colName]
  - df.iloc[rowIndex, colName]
- Access ***columns*** using square brackets. df[colName]
- Use df.values() to get a 2D Numpy array. There are only values without column names and row indexes.

```
dic = {'a':[1,2,3], 'b':[4,5,6], 'c':[7,8,9]}
df = pd.DataFrame(dic, index=['x', 'y', 'z’])

print("Access elements by row, df.iloc[0] = ", df.iloc[0])

print("Each row in DataFrame is a Pandas Series: ", type(df.iloc[0]))

print("Access elements by index, df.loc['x'] = ", df.loc['x’])

print("Access elements by column, df['a'] = ", df['a’])

print("Access single element df.iloc[1, 2] = ", df.iloc[1, 2])

print("Access multiple rows df.iloc[1:3] = ", df.iloc[1:3])

print("Access multiple columns, df[['a', 'b']]", df[['a', 'b']])
```

Have you installed TensorFlow or Pytorch?

- Any questions related to library installation?

A. Yes

B. No

Typical PyTorch model

- Model inherits from torch.nn.Module

A. Yes

B. No

PyTorch based model

- **Inheritance from torch.nn.Module**:
- PyTorch models typically inherit from the torch.nn.Module base class. This provides the model with the functionality to track and manage its parameters, as well as enabling nesting of modules.

PyTorch based model

- **__init__() function**:
- This is where the model's layers are instantiated. Any necessary data artifacts, such as vocabularies or pre-trained embeddings, may also be loaded during this phase.

PyTorch based model

- **forward() function**:
- The forward() method defines the actual computation that occurs when the model processes input data. It describes how the input passes through the layers to generate an output.
- Computational graph

A typical PyTorch optimization process

```
# get the inputs
inputs, labels = data

# zero the parameter gradients
optimizer.zero_grad()

# forward + backward + optimize
outputs = net(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

PyTorch optimization

- (optimizer.zero_grad()), compute the gradients with respect to the loss
- (loss.backward()), and then use those gradients to update the model parameters
- (optimizer.step()). This process is repeated for each batch of training data.

Pytorch

- **optimizer.zero_grad()**:
- This function is used to zero out the gradients of all the parameters that the optimizer will update.
- Gradients are accumulated by default in PyTorch, and calling zero_grad() ensures that the gradients from the previous iteration do not carry over to the current iteration.
- It is typically called before the backward pass (loss.backward()) in each training iteration.

Pytorch

- **loss.backward()**:
- This function is part of PyTorch's automatic differentiation system.
- It computes the gradients of the loss with respect to all the parameters in the model.
- It essentially performs the backpropagation algorithm, calculating how much each parameter contributed to the error in the prediction.
- After calling backward(), the gradients are stored in the .grad attributes of the model parameters.

Pytorch

- **optimizer.step()**:
- After the gradients have been computed with loss.backward(), this function is called to update the parameters of the model.
- The optimizer uses these gradients to adjust the model parameters according to the specified optimization algorithm (e.g., stochastic gradient descent, Adam, etc.).
- The learning rates, weight decay, and other hyperparameters are taken into account during this step.

Where is the location of PyTorch tensor by default?

- Suppose we have already installed NVIDIA-Driver and CUDA.

A. CPU

B. GPU

TensorFlow

- **Inheritance from tf.keras.Model**

In TensorFlow 2, what is the equivalent of PyTorch's forward method?

- Call()

TensorFlow 2 Optimization

```
# Define model, optimizer, and loss function
model = tf.keras.Sequential([...])
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

# Training loop
for epoch in range(num_epochs):
    for batch in train_dataset:
        inputs, targets = batch
        with tf.GradientTape() as tape:
            predictions = model(inputs)
            loss = compute_loss(targets, predictions)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variabl
```

TensorFlow2

- **trainable_variables**:
- trainable_variables is a property of a TensorFlow model that returns a list of all variables (tensors) in the model that are marked as trainable.
- These variables typically include weights and biases of layers that the optimizer adjusts during training.

TensorFlow2

- **gradient**:
- tf.GradientTape is a TensorFlow context manager that records operations for automatic differentiation.
- When you want to compute gradients with respect to certain variables, you open a tf.GradientTape context, perform your computations inside it, and then use it to calculate gradients.
- gradient(target, sources) computes the gradient of target with respect to sources using the recorded operations within the tf.GradientTape context.

TensorFlow2

- **apply_gradients**:
- optimizer.apply_gradients is a method used to update the model's trainable variables based on the computed gradients.
- It takes a list of (gradient, variable) pairs and applies the specified updates to each variable.

Where is the location of TensorFlow tensor by default?

- Suppose we have already installed NVIDIA-Driver and CUDA.

A. CPU

B. GPU